State-of-the-art

Reservoir Computing applied to *Audio Signal* with a focus on *Real-time Distortion*

Author : Timothy Lefebvre

Supervisor : Marvyn Gulina

State-of-the-Art Research Report



Faculty of Computer Science
University of Namur
Belgium
2025-2026

Abstract:

Reservoir Computing (RC) is a machine learning approach for modeling complex, non-linear dynamical systems. However, its application to real-time digital audio processing is severely constrained by algorithmic time complexity $\mathcal{O}(N)$. This literature review evaluates the feasibility of using RC as a black-box paradigm to emulate audio distortion under strict constraints: a standard 44.1 kHz sampling rate and a latency strictly below 15 ms. Starting from the foundational models of Jaeger and Maass Echo State Networks (ESNs) and Liquid State Machines (LSMs) we trace the evolution of the RC paradigm towards more computationally efficient architectures.

While the $\mathcal{O}(N^2)$ time complexity and memory capacity of early models makes them unsuitable for high-frequency audio streams, advanced architectures successfully bypass these limitations. Specifically, the Cascaded Delay-Line Reservoir Computer (DLRC) and the deep Time-Delay Reservoir Computer (deepTRC) provide an architecture achieving real-time emulation with algorithmic delays as low as 0.11 ms. Finally, we examine recent innovations, such as Next Generation Reservoir Computing (NG-RC), to identify future research directions for real-time audio signal processing.

Acknowledgements

I would like to express my deepest gratitude to my supervisor, Marvyn Gulina, for his guidance throughout this project. His benevolent supervision, his patience, and the clarity of his explanations have been of invaluable help in guiding me through the complex concepts of *Reservoir Computing*.

Contents

1 Introduction **1**
1.1 Structure of the paper . . . 1

2 Problem Statement **2**
2.1 Research Topic . . . 2

3 Methodology **2**
3.1 Literature Research . . . 2
3.2 Literature Selection . . . 3
3.3 Extraction and analysis . . . 4
3.4 Use of Generative AI . . . 4

4 State-of-the-art **5**
4.1 Keywords . . . 5
4.2 Core Concepts . . . 5
4.2.1 Inputs, Time series, signal . . . 6
4.2.2 Benchmarks . . . 7
4.3 Introduction . . . 8
4.4 Foundational Models ESN vs LSM . . . 9
4.4.1 The Echo state network . . . 9
4.4.2 Liquid State Machine (LSM) . . . 11
4.5 Enhancing Memory and Multi-Scale Dynamics . . . 12
4.5.1 Echo state networks with filter neurons and a "delay & sum" readout 13
4.5.2 Delay-line and single dynamical node . . . 16
4.6 Stacked and Deep Models . . . 17
4.6.1 Deep reservoir computing . . . 18
4.6.2 Cascade of Discrete-Time Delay Line-Based (DLRC) . . . 20
4.6.3 Deep Time-Delay Reservoir Computer (deepTRC) . . . 22
4.7 Emerging Paradigms and Recent Advancements . . . 23
4.7.1 Next Generation Reservoir Computing (NG-RC) . . . 23
4.7.2 The Volterra Kernel . . . 24
4.8 Bayesian Optimization of Hyper-parameters . . . 25

5 Conclusion **26**

1 Introduction

This state-of-the-art review focuses on the Reservoir Computing (RC) paradigm and its various modifications, with a specific emphasis on real-time audio and signal distortion. Specifically, we will explore which reservoir computing model is best suited for computing real-time audio distortion.

In the field of digital audio signal processing, modeling effects such as the distortion of a guitar amplifier or the restoration of non-linearly distorted signals requires capturing complex, highly non-linear, and time-dependent dynamics [16].

In this context, the **Reservoir Computing (RC)** paradigm represents a significant technological breakthrough. Introduced independently in the early 2000s by Jaeger under the names Echo State Networks (ESN) [13] and by Maass with his Liquid State Machines (LSM) [17], RC compared to other RNN drastically simplifies sequential prediction. Its core principle relies on a dynamic "reservoir" consisting of a recurrent network with fixed internal connections. This reservoir acts as a dynamic non-linear system that projects the temporal input signal into a spatial one while maintaining a fading memory of the input history. Training is restricted to the output layer (readout) using simple linear regression [13]. This ensures fast and computationally inexpensive learning that cannot get stuck in local minima [16].

Over the past years, Reservoir Computing models have evolved to compute signals faster and more accurately [5, 10, 2]. The integration of filter neurons (such as band-pass filters or leaky integrators) [13, 24] has enabled networks to synchronize multiple time scales inherent complex signal [13, 24]. More recently, Delay-Line Reservoir Computers (DLRC) [16] and their deep cascaded variants (deepTRC) [5] have replaced the heavy matrices of neurons with a single non-linear node coupled to a delayed feedback loop using time-multiplexing [10]. These models allow for lesser algorithmic computation times, finally making it possible to process, correct, and distort audio signals in strict real-time on standard computing hardware. [16].

This state-of-the-art review proposes discovering this paradigm in details. We will review the architectural foundations of Reservoir Computing, advanced optimization methods (such as Bayesian optimization of hyperparameters) to tune the dynamics of these models, and standard benchmarks used in literature (such as the Mackey-Glass system)[18].

1.1 Structure of the paper

This paper is structured as follows. Section 2 details the methodology used for this literature review, including our search queries and selection criteria. Section 3 formally defines the problem statement and explicitly states our core research questions regarding real-time constraints. Section 4 presents a comparative analysis of the state-of-the-art

Reservoir Computing architectures, systematically evaluating their suitability against our required performance and latency metrics. Finally, Section 5 concludes the review.

2 Problem Statement

The processing of non-linear signals inherently involves dealing with complex dynamic systems. In many contexts, audio recordings may suffer from unwanted non-linear distortion. These distortions modify the signal in a way that depends not only on the direct time-steps but often on the echo of the signal. [16].

The core research questions that will guide our comparative analysis throughout this study are formulated as follows:

- **To what extent can the Reservoir Computing paradigm satisfy the constraints of real-time audio processing regarding algorithmic complexity, latency, and predictive accuracy for modeling strongly non-linear audio distortion ?**
- **Among the existing RC architectures, which specific model provides the optimal trade-off to perform this task at a standard 44.1 kHz sampling rate, maintaining a latency strictly below 15 ms on standard computing hardware ?**

2.1 Research Topic

Even if reservoir computing topic is relevant in many application (real time signal distortion, AI, timeseries forecasting, chaotic series, etc.) we will here focus on the field of real time audio distortion.

3 Methodology

3.1 Literature Research

Initial scoping was guided by domain expert Marvyn Gulina [9, 8] and foundational literature [13]. To establish theoretical framework, the seminal papers defining the Echo State Network [13] paradigm and the work of University of Bordeaux [22] were analyzed first.

The literature review was conducted formally using primarily Google Scholar. To ensure a reproducible search process, my specific query was "reservoir computing real-time audio".

Following this, the research focused on peer-reviewed literature addressing the problem of real-time distortion correction. This targeted search highlighted **RC** applications,

notably the use of cascaded DLRC for guitar amplifier emulation [16]. By analyzing the reference lists of these highly relevant papers, other major architectural update within the RC paradigm were identified, such as the single dynamical node [2] and deep reservoir model [3]. Finally, subsequent searches were conducted to find recent models capable of handling complex input streams with reduced computational overhead, such as deep time-delay models [5].

3.2 Literature Selection

The selection of the literature was driven by several factors. First, I prioritized papers directly relevant to my field and research question, such as [16]. After this, I focused on foundational papers that introduced major structural modifications to the Reservoir Computing paradigm. The primary search engine I used here is **Google Scholar**, targeting reputable sources like *Nature Communications*, *Springer Nature*, *ScienceDirect*, *IEEE*. I also considered paper with at least 20 citation counts or those frequently referenced within the articles I analyzed.

Certain seminal articles were obvious choices for this literature review, such as [2], which had accumulated 2,133 citations, and [17], with 5,516 citations at the time of research.

After this I search for more recent papers under the **RC** paradigm with a focus on newer models that can compute real-time non linear distortion on more complex input stream or in faster rate such as [5, 4, 19]

My primary exclusion criterion is paper covering hardware and electronics implementation of **RC**. I encountered numerous articles discussing laser dynamics, satellite distortion, and electronic oscillators. Lacking a formal background in electronics, I chose to focus exclusively on software-based approaches to remain within my area of expertise.

My second exclusion criterion was based on publication dates, as I aimed to prioritize modern models within the RC paradigm. Consequently, I excluded papers older than the year 2000, which marks the beginning of the Reservoir Computing era with the work of Jaeger [13] and Maass [17].

The third exclusion criterion applied to biological studies. While Reservoir Computing falls under Artificial Intelligence, many papers discuss parallels with biological neurons. For obvious reasons, a biological review is beyond the scope of this work.

Finally, as Reservoir Computing involves complex mathematical frameworks, the equations presented in this paper are sourced directly from the referenced literature to ensure technical accuracy and rigor.

3.3 Extraction and analysis

The primary extraction and analysis methodology used in this review began with a reading of the articles. This initial step was followed by a study of the mathematical foundations underlying each computational paradigm. Then a comparative analysis of their experimental results.

3.4 Use of Generative AI

In accordance with institutional and academic integrity guidelines, this section details the use of AI. The Gemini model (Google) was strictly used to improve the quality of the text. This included orthographic and grammatical corrections, as well as stylistic reprocessing of certain passages to achieve a more formal and academic tone. The AI was used exclusively as a writing aid. All original ideas, research content, and analysis were developed solely by the author, and no content was generated directly from it.

4 State-of-the-art

4.1 Keywords

Reservoir computing · Timeseries · Echo state network · Signal distortion · real-time audio processing · deep reservoir computer · delayed-feedback

4.2 Core Concepts

Before exploring many Reservoir computing models and looking for the better suited for our needs, we will first define some core concepts.

General Time Series Processing refers to the computational analysis of data indexed in chronological order. This includes any signal that evolves over time—such as sinusoids, audio signals, or stock market fluctuations—where each data point at time t is analyzed in relation to its preceding $(t-1)$ and succeeding $(t+1)$ states.

Reservoir computing is a mathematical paradigm belonging to the machine learning field, composed of an input-dependent dynamical system (the reservoir) and a readout trained to compute the desired output from the reservoir state.

This definition is quite hard to produce and maybe not the best since in the literature we often find that reservoir computing is the collectively known mix between ESN and LSM.[11].

Some say it is based upon the idea of temporal signal processing on fixed weights.[24].

In fact, to the best of my knowledge, no official definition was given and reservoir computing is often defined as the ESN[13] approach of Jaeger. [3].

Nonlinearity "is a property of mathematical functions or data that cannot be graphed on straight lines, systems whose output(s) are not directly proportional to their input(s), objects that do not lie along straight lines, shapes that are not composed of straight lines, or events that are shown or told out-of-sequence." definition from Wikipedia [1].

Fading memory refers to the property of a Reservoir Computing (RC) system to gradually "forget" its initial conditions[13]. As the reservoir processes an input stream, the influence of past states decays over time, ensuring that the system's internal dynamics depend primarily on the most recent history of the signal.

Time dependence Mathematically, the **hidden state** $(x(t))$ of the reservoir is defined as a function of the entire history of previous timesteps $\{u(t), u(t-1), \ldots, u(t-n)\}$ provided to the system [19].

The **Echo State Property (ESP)** is the fundamental mathematical requirement that bridges these concepts. In essence, the ESP is the combination of a fading memory

and the functional dependence of the state on the input: it ensures that hidden states disappear by feeding it in new data, allowing the current dynamics to be driven solely by the continuous input stream.

4.2.1 Inputs, Time series, signal

For the purpose of this paper, the terms *input*, *time series*, *signal*, and *sound* are treated as **interchangeable**. We define these as **discrete-time** datasets [27], represented as a sequence of observations where each point is indexed by a specific time step.

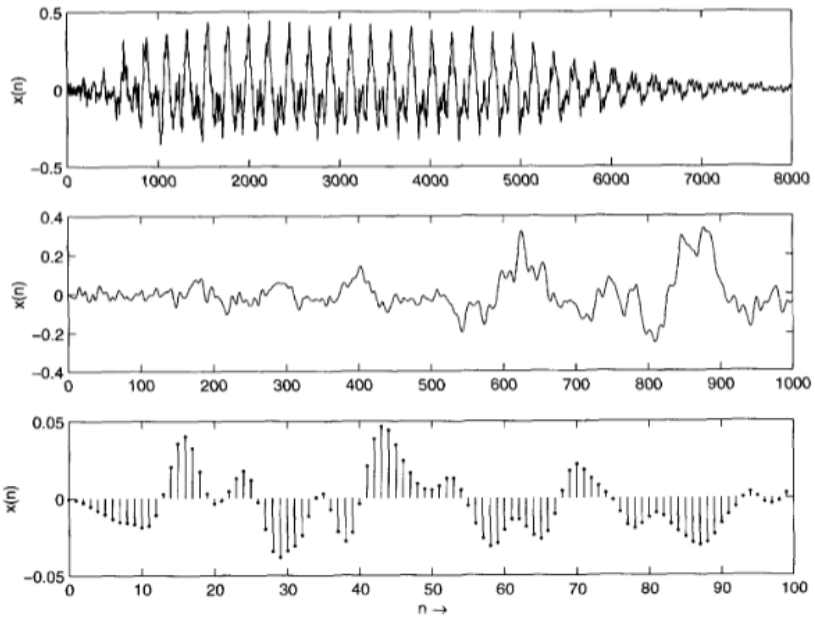


Figure 1: Different time representations for digital audio signals [27]

A standard computer typically processes audio signals at a **sampling frequency** of **44.1 kHz** [27], which means we have 44,100 data points (or samples) per second.

Each sample's amplitude is quantized and stored as a fixed number. For a **16-bit depth** representation, this corresponds to $2^{16} = 65,536$ possible values, included in [-32,768; 32,767] [27]. Stereo quality implies that all those measurements are taken twice (one sequence for the left channel and one for the right).

The audio is recorded by an **ADC (Analog-to-Digital Converter)**, which samples the continuous wave into bits stored inside the computer. It is then processed and sent to a **DAC (Digital-to-Analog Converter)** to reconstruct an analog signal [27] that is as close as possible to the human ear's perception, though never perfectly identical to

the original.

Today, ADC and DAC are usually integrated directly into the computer's motherboard.

In order to keep our computation within a delay that the human ear cannot recognize, we must respect the following delays:

- **Less than 15ms**: the human ear doesn't hear a temporal delay [27].
- **Between 10 and 25ms**: perceived as a chorus effect (a form of superposition of the same signal but repeated with echo) [27].
- **More than 50ms**: latency is clearly perceived by the human ear [27].

4.2.2 Benchmarks

We will also define some of the most widely used benchmarks in the reservoir computing field.

First, the most common benchmark found in the literature [13, 11, 24, 5, 8, 9] is the **Mackey-Glass system**. Originally introduced in the field of medical biology, this chaotic time series is named after Michael Mackey and Leon Glass from their seminal 1977 paper [18]. The dynamics of the Mackey-Glass is governed by a nonlinear delay differential equation, defined as:

$$\frac{dy(t)}{dt} = \frac{\alpha y(t-\tau)}{1+y(t-\tau)^{\beta}} - \gamma y(t) \tag{1}$$

where $y(t)$ is the state variable, τ is the time delay and α, β, and γ are the system parameters.

We must keep in mind that the greater the delay τ, the harder it becomes to predict the next value.

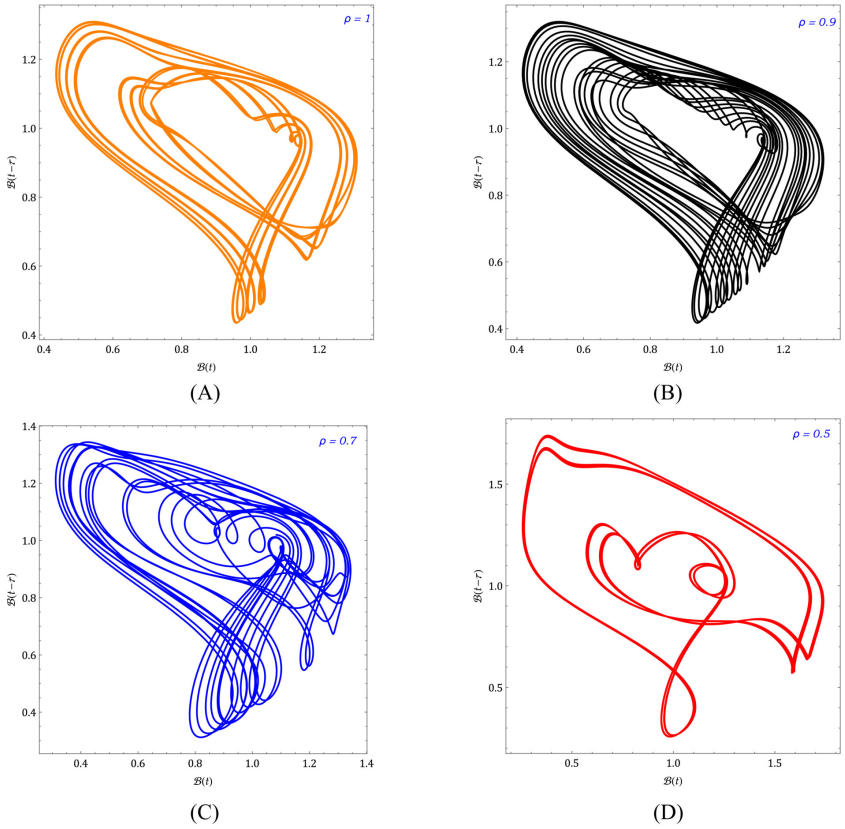


Figure 2: Parametric nature of the system for varying ρ ((A) 1, (B) 0.9, (C) 0.7, and (D) 0.5) combined with $\kappa = 10$, $\tau = 1.5$, $\mu = 1$, and $\alpha = 2$ [26].

Even though the Mackey-Glass system does not represent real-time audio distortion, it is still a benchmark used in almost every paper reviewed for this state-of-the-art. In fact, it shares several properties with real-time audio forecasting: it is a **chaotic, non-linear system**, and it is **past-dependent**. Because of these similarities, we will use it in this paper as an indicator of a model's ability to compute the desired distortion.

Another widely used benchmark is **spoken digit recognition** [2, 20]. Although it also relies on audio signals, it is fundamentally a classification task rather than a predictive one, which is why it will not be covered here.

4.3 Introduction

We will now look for the best variation of the reservoir computing paradigm for real time audio distortion. we will study latest and major reservoir computing model and hopefully find the best for real time audio distortion computation.

Since we will focus on real-time distortion and on standard hardware (consumer laptop

without powerful GPU), time complexity $O(n)$ will be focus in our search.

4.4 Foundational Models ESN vs LSM

Several reservoir computing architectures have been proposed, including the Echo State Network (ESN) and the Liquid State Machine (LSM), which share similar core principles but differ in their specific implementations. They are widely recognized in the literature as the foundational models of the **RC** paradigm.

4.4.1 The Echo state network

The Echo State Network (ESN), introduced by Herbert Jaeger [13], belong to the **RNN** family [14].**ESN** relies on the concept that a large, non-linear reservoir can provide the rich dynamics and "echo states" required for signal prediction and modeling. Crucially, these states are mapped to the output by a readout layer that is trained using simple linear regression. This work stands as the foundational paper for ESNs.

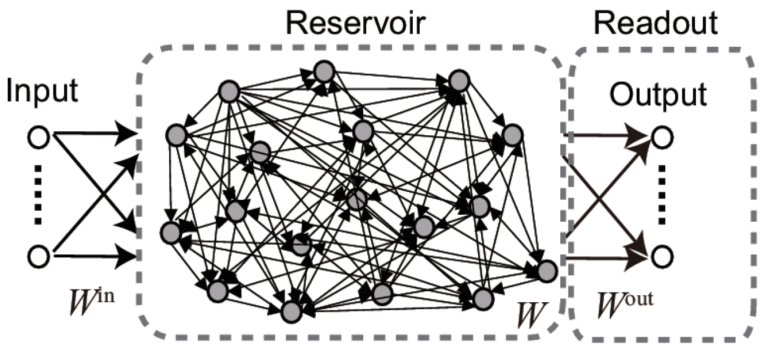


Figure 3: Overview of a reservoir computer [23]

As shown in Figure 3, this architecture is based on three major components:

- **An input layer:** This feeds the sequential data into the reservoir. Naturally, the time-series data must be ordered chronologically and feed from the oldest to the most recent.
- **A reservoir:** Its primary function is to project the input into state space. Through its fixed recurrent connections and non-linear activation functions, it creates a rich set of dynamics (often referred to as **interpretable chaos**).
- **A readout layer:** The readout weights are the only trained matrix in this model and generally in the broader Reservoir Computing paradigm, since the internal reservoir connections remain fixed.

The idea here is simple: first, we initialize the reservoir's connections using distribution law (with respect over spectral radius) without touching them ever again. Then, we feed the reservoir with an input sequence, and finally, we use linear regression to define the optimal weights for the readout to decode the complex dynamics of the reservoir and get the desired output. "This implies that the training is fast, simple and cannot get stuck in local minima (unlike other training methods)" [11].

ESN is characterized by the recurrent feedback loop in their synaptic connection. [14] giving him dynamic memory.

The first hidden state equation given by Jaeger is :

$$\mathbf{x}(n+1) = f(\mathbf{W}_{in}\mathbf{u}(n+1) + \mathbf{W}\mathbf{x}(n) + \mathbf{W}_{back}\mathbf{y}(n)) \tag{2}$$

with:

- $\mathbf{x}(n)$ is the N-dimensional state vector of the reservoir.
- $\mathbf{u}(n+1)$ is the external input signal at time $n+1$.
- $\mathbf{y}(n)$ is the output signal of the previous step.
- $\mathbf{W}_{in}$ is the input weight matrix.
- $\mathbf{W}$ is the fixed recurrent weight matrix of the reservoir.
- $\mathbf{W}_{back}$ is the optional output feedback weight matrix.
- f is the nonlinear activation function of the reservoir nodes, typically the hyperbolic tangent (tanh).

Once we compute the internal states $\mathbf{x}(n)$, the optimal readout weights can be computed offline using a standard Ordinary Least Squares (OLS) linear regression:

$$\mathbf{W}_{out} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y}_{target} \tag{3}$$

To achieve stability, the reservoir's weight matrix is scaled so that its **spectral radius** ($\rho(W)$) is maximal (singular or eigen) value, is strictly less than one.

The spectral radius is mandatory to keep the **ESP**.

The mathematical implications of the spectral radius are explained in more detail by Jaeger [13] and Yperman [25].

As always the ESN, as its name implies, and like any other echo model, is the **Echo State Property (ESP)** defined previously.

Jaeger defines the **leaky integrator neuron model**, which is used to learn slower dynamical systems (like slow sinusoids). The idea here is to slow down the internal

dynamics of the reservoir to stabilize it for slower timescales [13]. Leaky integrator neurons can also maintain the dynamics of a reservoir if the input stops abruptly. The evolution of a continuous-time leaky integrator network is governed by the following differential equation:

$$\dot{\mathbf{x}} = C\left(-a\mathbf{x} + f(\mathbf{W}_{in}\mathbf{u} + \mathbf{W}\mathbf{x} + \mathbf{W}_{back}\mathbf{y})\right) \tag{4}$$

where C is a global time constant, a is the leaking decay rate, and f is the nonlinear activation function (typically tanh) [13].

Jaeger pursues optimization and conducts further experiments on leaky-integrator neurons in [15] by using a gradient descent loss function over hyperparameters. However, as this paper focuses more on slow dynamics and classification, it has little to no added value for this state of the art.

We can also note that Jaeger provides another, simpler paper regarding ESN implementation, which includes various numerical examples. [12] This paper is intended to be a 'gentle' and 'detailed' guide for the Echo State Network approach. It features practical demonstrations, such as a sinusoid generator, to illustrate the ESN dynamics and training process.

It is also worth noting, standard **ESN** was successfully used by jaeger in [14] to reduce error-rate signal in cellular communication, emulating the older volterra DFE system in use in that period.

As we will see later in this paper, Even if standard ESNs are an effective paradigm for modeling non-linear distortion, they are computationally demanding. The $O(N^2)$ complexity required for the matrix multiplications at each time step makes large ESNs impractical for real-time audio distortion processing on standard hardware (CPU or GPU).

Other limitations of the **ESN** include difficulties in learning patterns based on multiple timescale signals, as well as a significant **warm-up** period, defined as the minimal data required to detach the inner state of the reservoir from its initial one. In our specific case, the short-term memory of the ESN is strictly limited by the size of the reservoir. Since real-time audio processing is computed at higher frequency, all these limitations becomes major. We will discuss the solution proposed by Holzmann and Hauser [11] later.

4.4.2 Liquid State Machine (LSM)

Developed in parallel to Jaeger work on Echo State Networks (ESN) [13], Wolfgang Maass introduced the **Liquid State Machine (LSM)** [17]. Both models share the same fundamental Reservoir Computing paradigm: they rely on the **ESP**. In both approaches, the input is fed into a fixed, non-linear dynamical system, and the resulting states are decoded by a readout layer using linear regression [13, 17].

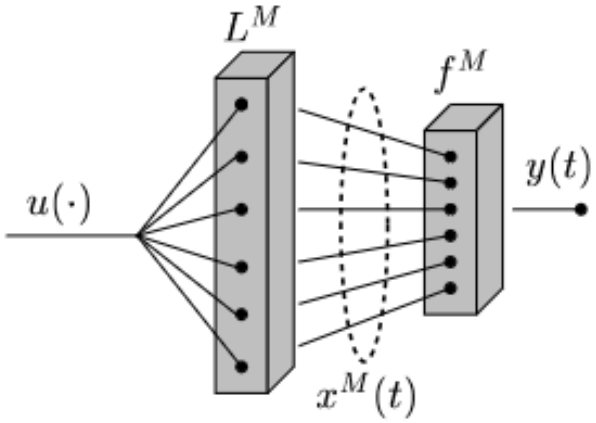


Figure 4: Liquid state machine [17]

Even if ESNs and LSMs are conceptually identical in their architecture, LSMs typically employ spiking neurons to compute the hidden state, whereas ESNs use discrete-time neurons.

These spiking neurons get impulses from the previous neurons connected to them, sum them and keep the signal internally, then let the signal fade.

If the neuron gets to a certain maximum point, it discharges itself instantly.

A major distinction that is particularly useful for us lies in the training approach. The standard Jaeger ESN typically uses *offline training*: the entire input sequence is fed into the reservoir to collect all valid states after an initial warm-up (washout) period, followed by a single global linear regression to compute the output weights. Whereas the Maass LSM framework naturally accommodates *online training* [17]. This allows the readout weights to be adjusted iteratively and locally at specific time steps as the system evolves, enabling continuous adaptation to new data without the need to store the entire state history.

Similar to the Echo State Network (ESN) by Jaeger [13], the Liquid State Machine (LSM) suffers from an algorithmic complexity of $\mathcal{O}(N^2)$. This make it unsuitable for real-time audio distortion processing.

4.5 Enhancing Memory and Multi-Scale Dynamics

In order to overcome the inherent limitations of foundational models regarding their computational complexity and restricted memory capacity, several architectural upgrades have been proposed. The first modifications we will cover are the integration of Infinite Impulse Response (IIR) filter neurons and the Delay & Sum readout, which were specifically designed to significantly enhance the short-term memory capacity and manage long-term dependencies [11].

Furthermore, we will introduce the single nonlinear dynamical node with delayed feedback, often referred to as Delay-Line Reservoir Computing. This approach represents a major architectural breakthrough, as it successfully reduces the algorithmic time complexity per sample from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$ [2].

4.5.1 Echo state networks with filter neurons and a "delay & sum" readout

In 2010, Holzmann and Hauser [11] improved Jaeger original ESN architecture [13] by extending the filter neurons proposed by Wyffels et al. [24] to arbitrary Infinite Impulse Response (IIR) filters,[11] and by introducing a new delayed readout mechanism. These modifications are particularly suited for complex tasks like real-time audio distortion.

This approach was purposely designed to overcome the short-term memory limitations associated with reservoir size when learning from the distant past. Furthermore, it addresses the strong coupling between neurons that makes it almost impossible for standard ESNs to learn signals composed of multiple attractors or frequencies [11, 16].

They introduced two major improvements over the original ESN:

First, they integrated arbitrary **IIR (Infinite Impulse Response)** filters directly into the reservoir neurons.

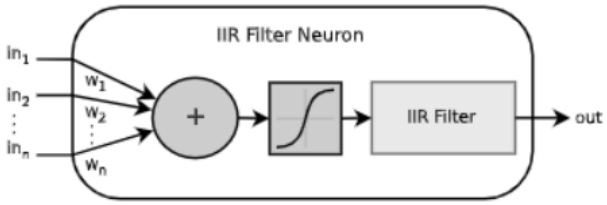


Figure 5: IIR Filter neuron [11]

The main objective of these **IIR** filters is to specialize neurons to different frequency bands, enabling the ESN to successfully model multiple attractors.

This filter is placed immediately after the nonlinear activation of every reservoir cell. Consequently, the system performs a two-step computation. First, the standard reservoir update determines an intermediate state $\tilde{x}_i(n+1)$.

$$\tilde{x}_i(n+1) = f\left(\rho \mathbf{W}_i \mathbf{x}(n) + \mathbf{W}_{in,i}\mathbf{u}(n+1) + \mathbf{W}_{fb,i}\mathbf{y}(n) + \upsilon(n+1)\right) \tag{5}$$

where $\mathbf{W}_i$, $\mathbf{W}_{in,i}$, and $\mathbf{W}_{fb,i}$ represent the i-th rows of the respective weight matrices, ρ is the spectral radius, and f is the nonlinear activation function.

Subsequently, the IIR filter is applied to this intermediate value to produce the final state of the cell, $x_i(n+1)$.[11]:

$$x_i(n+1) = \frac{1}{a_{i,0}} \left(\sum_{j=0}^{K} b_{i,j} \tilde{x}_i(n-j) - \sum_{j=1}^{K} a_{i,j} x_i(n-j) \right) \tag{6}$$

where:

- $b_{i,j}$ are the fixed weights of the feedforward path to neuron i.
- $a_{i,j}$ are the fixed weights of the feedback path t neuron i.
- K determines the order of the digital filter. (how deep neuron keep information)

To maintain the **Echo State Property (ESP)** in this extended architecture, specific stability conditions must be met: as always the spectral radius ρ of the internal weight matrix must remain within $0 \leq \rho < 1$, **and** the maximum gain of each individual IIR filter must not exceed 1 [11].

Because the parameters for these filters are fixed in advance and specific to each neuron, they do not require additional training, thus preserving the computational efficiency of standard ESNs [11].

Secondly, they added a **delay&sum (D&S) readout**.

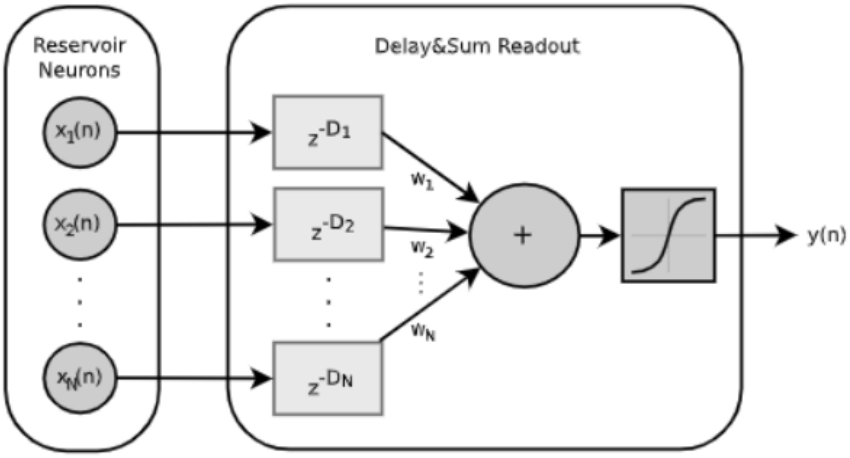


Figure 6: Delay&Sum readout [11]

This **D&S readout** is a major upgrade to the ESN, designed to drastically increase the memory capacity of the network. Until now, the memory of the reservoir was strictly limited by its size (the number of neurons) [13]. The delay&sum mechanism computes the optimal temporal delay needed for each synaptic connection to the output, minimizing the

loss and allowing the network to retain past information much longer than a traditional reservoir.

Assuming an ESN with N internal units and L inputs, the reservoir and input signals are combined into a single vector $\mathbf{s}(n) = (x_1(n), \ldots, x_N(n), u_1(n), \ldots, u_L(n))^T$. The m-th output $y_m(n)$ of the D&S readout is computed as:

$$y_m(n) = g\left(\sum_{i=1}^{N+L} W_{i,m}^{out} s_i(n - D_{i,m})\right) \tag{7}$$

where:

- $D_{i,m}$ is the integer delay applied to the signal s_i before reaching the m-th output.
- $W_{i,m}^{out}$ is the trainable output weight.
- g is the activation function of the output neuron.

In addition to training the standard readout weights, the delay&sum readout requires training to find the optimal temporal delay for each connection.

A method proposed by the authors is to estimate the cross-correlation $\hat{r}$ between each internal signal $s_j(n)$ and the target output $t_m(n)$:

$$\hat{r}_{s_j t_m}(\tau) = \frac{1}{n_{max}} \sum_{n=0}^{n_{max}-1} s_j(n) g^{-1}(t_m(n+\tau)) \tag{8}$$

where n_{max} is all the information. The optimal delay $D_{j,m}$ for each connection is then chosen as the time shift τ that maximizes the absolute correlation:

$$D_{j,m} = \arg\max_{\tau} |\hat{r}_{s_j t_m}(\tau)| \tag{9}$$

Once all delays are estimated and fixed, the reservoir and input signals are shifted in time accordingly. The output weights W^{out} are then optimized using a standard offline linear regression on these correctly delayed signals [11].

Holzmann and Hauser proposed two different algorithms for training the **D&S** readout to find the optimal delay for each neuron, which will require further investigation.

These modifications to the original **ESN** make this model a strong competitor for real-time audio distortion processing. Although this architecture is a major upgrade in the **RC** paradigm, some limitations remain for our objective. First, the computational complexity is still $\mathcal{O}(N^2)$, and as the authors themselves stated, if we try to learn a single attractor instead of multiple ones, the **IIR** filters perform worse [11].

4.5.2 Delay-line and single dynamical node

In 2011, Appeltant et al. [2] introduced a major architectural improvement to the Reservoir Computing paradigm. Their main objective was to drastically reduce the computational complexity of traditional Echo State Networks. To achieve this, they proposed replacing the large pool of physically interconnected neurons with a single nonlinear node subjected to a delayed feedback loop.

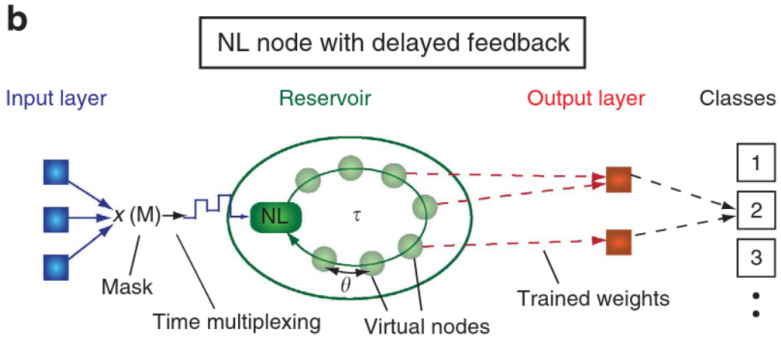


Figure 7: Single nonlinear node with delayed feedback [2]

In the **DLRC** (or Single Node) approach, the node receives its own previous output after a delay time τ.

It rely on 3 major steps, First **the masking**: Where the discrete-time input sequence $u(n)$ feed the reservoir during full delay period τ. It is then multiplied by a periodic masking signal $M(t)$ consisting of N constant values. The resulting continuous-time masked input $J(t)$ fed into the node [2]:

$$J(t) = M(t)u(n) \quad \text{for } t \in [n\tau, (n+1)\tau] \tag{10}$$

where the mask $M(t)$ changes value at fixed intervals $\theta = \tau/N$.

Then the **masked signal is injected** into the nonlinear node. The update rule for the node state $x(n)$ at discrete time n using Infinite Impulse Response (IIR) [24]:

$$x(n) = ax(n-1) + (1-a)f\big(\gamma J(n) + \eta x(n-N)\big) \tag{11}$$

where:

- $x(n)$ is the current state of the system.
- $x(n-N)$ is the delayed feedback state from the previous cycle.
- $a \in]0,1[$ is the filter factor controlling the inertia of the node and the mixing of adjacent virtual states.
- γ is the input strength.
- η is the feedback strength.

- f is the nonlinear activation function.

And finally the linear regression take place like define by Jaeger[13].

To achieve optimal performance, several crucial parameters governing the dynamics of this temporal reservoir must be carefully tuned:

- **Feedback strength** (η): Controls the influence of the past delayed state on hidden state. It serves a role analogous to the spectral radius in a traditional RC, maintaining the system at the edge of stability.
- **Input gain** (γ): Determines how strongly the nonlinear node is driven by the external input signal, which is analogous to the input scaling in an ESN.
- **Delay time** (τ): The total duration of the feedback loop. It is directly related to the total number of virtual nodes in the system.
- **Node separation** (θ): The constant time interval between two consecutive virtual nodes in the delay line ($\theta = \tau/N$). Its value determines the mixing of adjacent states, acting similarly to the sparsity of the interconnection matrix in a standard reservoir.

This temporal multiplexing approach allows the system to process time-series data efficiently while significantly reducing the computational cost. By replacing heavy matrix multiplications of a standard ESN with a simple time-delay loop, the computational complexity drops from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$ [16] per time step. As discussed later in this paper, a specific discrete-time modification of this paradigm was successfully implemented by Keuninckx et al. [16] to model real-time audio distortion.

4.6 Stacked and Deep Models

In this section, we will define models under the (RC) paradigm that are composed of multiple reservoirs or layered architectures, such as Delay-Line Reservoir Computing (DLRC) [2], Cascaded DLRC [16], and Deep Time-Delay Reservoir Computing (DeepTRC) [5].

These advanced models emerge as the most suitable candidates to address our research questions. The core principle behind these architectures is to overcome the final computational barriers between the RC paradigm and strict real-time audio processing constraints by:

- **Proposing a hierarchy of timescales:** Layered architectures inherently develop a multiple time-scale organization, allowing the network to naturally capture both short- and long-term temporal dependencies across different levels of abstraction [3].
- **Enhancing dynamical richness:** Stacking reservoir layers significantly amplifies the richness of the state dynamics, increasesing memory capacity without the need for wider, fully connected networks [3, 5].

- **Reducing algorithmic time complexity:** By utilizing single nonlinear nodes with delayed feedback and unidirectional coupling, these architectures drastically reduce the computational cost. This design provides a constant, low computation time even as the total number of virtual nodes increases, effectively bypassing previous bottlenecks and making real-time processing achievable on standard hardware [2, 16, 5].

4.6.1 Deep reservoir computing

In 2017, Gallicchio et al. [3] explored the concept of Deep Reservoir Computing, bridging deep learning architectures with the reservoir computing paradigm.

Their idea was to stack multiple reservoirs one on top of the other:

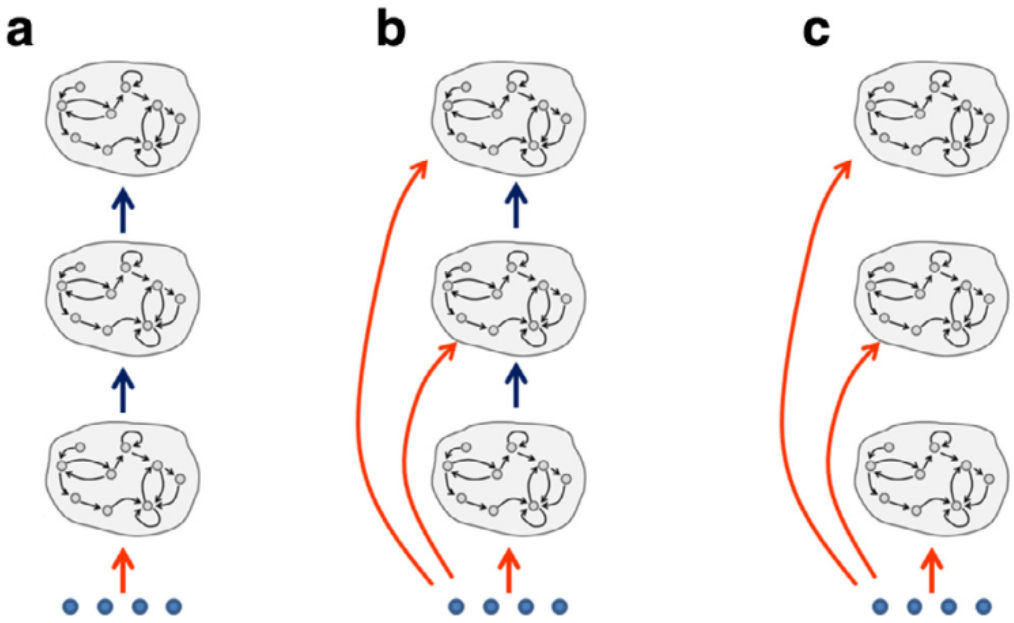


Figure 8: Deep RC architectures [3]

For this purpose, they developed three distinct models based on the ESN paradigm but incorporating deep layers:

- **DeepESN**: (a) The simplest deep architecture, where each reservoir layer directly feeds into the next one.
- **DeepESN-IA** (b) (Input to All): Every layer in the stack receives the external input signal directly.

- **GroupedESN**: (c) Each sub-reservoir receives the external input directly without feeding into the others (they are arranged in parallel rather than stacked).

The experiments conducted on these new models revealed that the DeepESN architecture is better suited than a standard ESN processing complex temporal data. It develops hierarchical memory, capturing rapid dynamics (short-term memory) in the first layer and progressively slower dynamics (long-term memory) in the deeper layers. This multiple time-scale representation is highly relevant for our research on real-time audio distortion, as such complex audio signals inherently require modeling both shorter and longer temporal dependencies simultaneously.

Since the benchmarks in [3] indicate that DeepESN is the only architecture possessing added value for our specific application (short and long term memory), and because the DeepESN-IA and GroupedESN models were primarily introduced as baselines to isolate the effects of layering, these two models will not be explored further in this study.

First, the first reservoir is fed using the external input sequence and each successive layer is fed directly by the output state of the previous one. The input $\mathbf{i}^{(l)}(t)$ for the l-th layer at time step t is thus defined as [3]:

$$\mathbf{i}^{(l)}(t) = \begin{cases} \mathbf{u}(t) & \text{if } l = 1 \\ \mathbf{x}^{(l-1)}(t) & \text{if } l > 1 \end{cases} \tag{12}$$

the internal state $\mathbf{x}^{(l)}(t)$ of each layer l is updated at each discrete time step t according to the following state transition function [3]:

$$\mathbf{x}^{(l)}(t) = (1 - a^{(l)})\mathbf{x}^{(l)}(t-1) + a^{(l)} \tanh\left(\mathbf{W}_{in}^{(l)}\mathbf{i}^{(l)}(t) + \hat{\mathbf{W}}^{(l)}\mathbf{x}^{(l)}(t-1) + \boldsymbol{\theta}^{(l)}\right) \tag{13}$$

where:

- $a^{(l)} \in [3]$ is the leaky parameter for layer l, controlling the speed of its dynamics.
- $\mathbf{W}_{in}^{(l)}$ is the initialized input weight matrix for layer l.
- $\hat{\mathbf{W}}^{(l)}$ is the recurrent weight matrix of layer l.
- $\boldsymbol{\theta}^{(l)}$ is the bias-to-reservoir weight vector for layer l.

Then the output weight are computed using a linear regression, the author also mention de Ridge regression to prevent overfitting.

To compute the final output, once training is complete, The network's output $\mathbf{y}(t)$ is computed as a linear combination of all layers [3]:

$$\mathbf{y}(t) = \mathbf{W}_{out}[\mathbf{x}^{(1)}(t), \mathbf{x}^{(2)}(t), \ldots, \mathbf{x}^{(N_L)}(t)]^T + \boldsymbol{\theta}_{out} \tag{14}$$

where $\mathbf{W}_{out}$ is the readout weight matrix and $\boldsymbol{\theta}_{out}$ is the bias-to-readout vector [3].

The other experiments discussed in [3] involve the use of **Intrinsic Plasticity (IP)**, sometimes referred to as homeostatic plasticity. This concept was formally adapted to the RC paradigm by Schrauwen et al. [20], extending the foundational work of Triesch [21].

To be completely honest, understanding the IP in detail requires a deep understanding of biology and mathematics that goes way beyond me. From what I understand, essentially, IP is a local, learning rule that allows each individual neuron to fine-tune the gain and bias of its own activation function. By forcing the neuron's output to follow a target probability distribution, the rule maximizes the entropy within the reservoir. This ensures the neurons do not saturate.

As summarized by the authors:

> "The simple, local IP rule effectively makes reservoirs significantly more robust: it empowers the reservoirs with the ability to autonomously and robustly adapt their internal dynamics, irrespective of initial weight setting, input scaling or topology, to a dynamic regime which is suited for the given task. The rule is purely input driven, and adapts the reservoir in an unsupervised way." [20].

4.6.2 Cascade of Discrete-Time Delay Line-Based (DLRC)

In 2017, Keuninckx, Danckaert, and Van der Sande [16] revisited the single dynamical node concept originally defined by Appeltant et al. (2011) [2] and expanded upon it by integrating the IIR filter neuron approach proposed by Holzmann [11]. While the standard DLRC was effective for simpler tasks, although identified significant limitations when applied to the complex dynamics of a guitar amplifier. This specific task requires both a long memory (for the acoustic response) and a strong nonlinearity ,requirements that a standard ESN or a single-node DLRC could not solve alone.

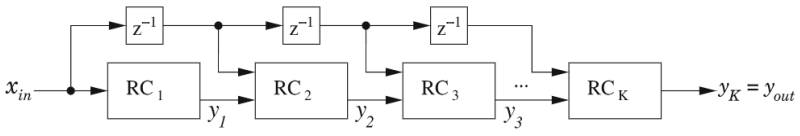


Figure 9: Cascade of discrete-time delay line reservoir computer [16]

To overcome this, Keuninckx proposed a novel architecture: a cascade of k reservoirs, each containing 100 virtual nodes[2]. The 'genius' of this approach lies in its multiplexing strategy, passing both the original guitar signal x_{in} and the previous layer output y_{i-1} to the next reservoir. The use of filtered virtual neurons [2] to match audio timescales. This structure successfully emulated the rich sound of guitar amplifier in real-time on standard hardware (a consumer laptop), demonstrating the ability of reservoir computer paradigm for nonlinear audio signal real-time distortion at 44.1 kHz.

To process the audio signal effectively, their discrete-time architecture relies on the use of an **input mask** [2, 16]. The mask is defined as a periodic sequence of size N, where N is the number of virtual nodes composing the feedback loop. At each time interval, the incoming audio sample is multiplied by the mask values. This crucial step distributes the signal across the virtual nodes, maintains the entropy inside the reservoir, and keeps the dynamics in a "transient state"[16], preventing the system from relaxing into a stable equilibrium.

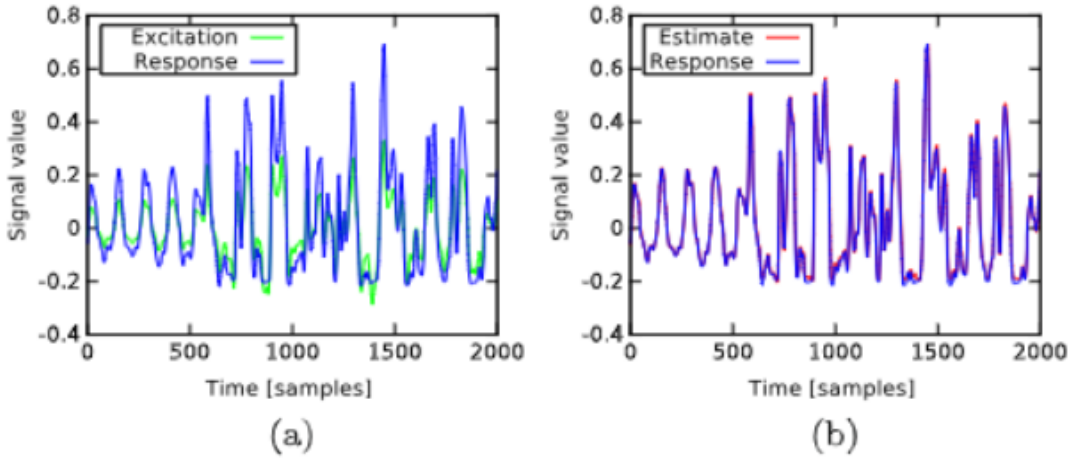


Figure 10: comparison of single vs cascade delayed feedback
[2]

The results displayed in the upper graph illustrate a precision comparison between the **Cascaded delayed feedback** architecture [16] and the standard **DLRC** [2]. They demonstrate a clear superiority of the cascaded architecture, showing a near-perfect match between the expected and obtained results.

To the best of our knowledge, Keuninckx and his team are the only researchers to have conducted real-time audio distortion tests at 44.1 kHz with such low latency. Although they did not explicitly state a global latency time in milliseconds, their architecture introduces an algorithmic delay of exactly one sample per reservoir unit [16]. For a 5-unit cascade at 44.1 kHz, this corresponds to a delay of merely 5 samples (approximately 0.11 ms), which is far below the 15 ms previously defined.

This exceptional real-time performance and precision make this architecture a clear choice for further testing on other real-time audio distortion tasks.

At this point, our reading confirmed that some models under the Reservoir Computing

paradigm is viable for real-time audio distortion, but the relevant paper being somewhat dated (2017). Therefore, the remainder of this state of the art focuses on the newest optimizations of the paradigm, such as the Next Generation Reservoir Computer [4] and Deep Time-delay reservoir computer. We cover in the next chapter to determine if these new models can achieve better results than the methods previously established.

The work produced by Gallicchio et al. [3] represents a significant step forward in our search for a model under the **RC** paradigm capable of real-time audio processing. The DeepESN architecture defined there, with its ability to simultaneously capture both short and long memory patterns, will be precious in this regard.

4.6.3 Deep Time-Delay Reservoir Computer (deepTRC)

In 2020, Mirko Goldmann and his team introduced a new model [5] based on the foundational works of Jaeger [13], Appeltant [2], and Gallicchio [3].

This new **deepTRC** architecture is a hybrid approach that merges **Deep Reservoir Computing** [3] with the **DLRC**) paradigm [2].

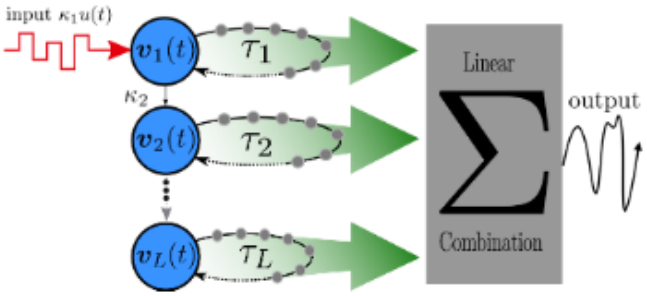


Figure 11: Deep time-delay reservoir computer [5]

In this architecture, the external input signal is fed exclusively to the first layer. This initial layer processes the input signal using the nonlinear delay node model proposed by Appeltant [2].

This unidirectional process continues sequentially for every layer in the model.

Once the continuous state of a layer is computed, it is multiplied by a coupling gain and is instantaneously used to feed the next layer. It is worth noting that while the external input gain applied to the first layer is often greater than one, the inter-layer coupling gains are typically smaller, acting as linear filters.

Once the initial warm-up (washout) period is completed and the entire input sequence has been fed, the states from all virtual nodes across all layers are extracted. These combined states are then used simultaneously to train the readout weights with single global linear regression.

Even if this model was not directly tested on real-time audio distortion processing, this architecture has strong similarities with other models previously studied and is reduce warm-up phase[8].

Consequently, the Deep Time-Delay Reservoir Computing **(deepTRC)** architecture emerges as a highly promising candidate for subsequent testing phases. By leveraging a layered topology, it inherently benefits enhanced short-term memory capacity and greater richness of state dynamics, sharing fundamental properties with previous models [16, 3, 2].

Furthermore, because its architecture closely resembles the cascaded DLRC approach [16], the deepTRC is expected to achieve comparable real-time processing capabilities without prohibitive computational overhead. Indeed, its algorithmic time complexity scales linearly with the total number of virtual nodes (L layers $\times N$ nodes per layer), strictly bypassing the $\mathcal{O}(N^2)$ bottleneck of classical **ESN**. Combined with its proven high predictive accuracy on chaotic benchmarks [5], this optimal trade-off between dynamical richness and computational efficiency makes it a particularly strong model to address our specific research questions.

4.7 Emerging Paradigms and Recent Advancements

This final section reviews the most recent advancements in the literature, specifically the Next Generation Reservoir Computing (NG-RC) [4] and the Volterra kernel [6].

These latest models under the **RC** paradigm will be examined to evaluate their potential to overcome the limitations of previously discussed architectures. Any significant improvements over earlier models will be assessed, with a particular focus on reductions in algorithmic time complexity, enhancements in memory capacity, and increased predictive precision.

4.7.1 Next Generation Reservoir Computing (NG-RC)

The **Next Generation Reservoir Computing (NG-RC)**, which is mathematically identical to Nonlinear Vector Autoregression (NVAR), has been proven to be equivalent to a linear RC [4]. It excels at standard benchmark tasks while offering similar or better performance compared to traditional ESN.

The core concept here is to eliminate the reservoir. Instead of broadcasting the input into reservoir, the NG-RC creates a feature vector directly from time-delayed observations of the input data and their nonlinear functionals [4]. The output weight matrix is then applied directly to these features.

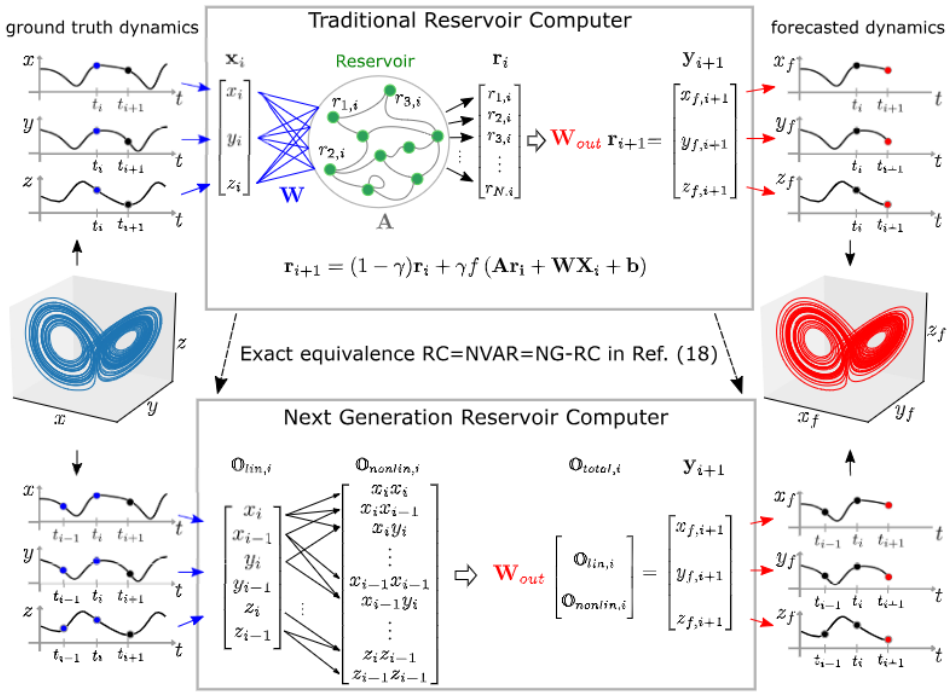


Figure 12: Exact equivalence between a traditional RC and the NG-RC architecture [4]

According to the authors, **the NG-RC requires a dramatically shorter warm-up period** [4]. This property has two significant impacts. First, it **decreases required length of datasets**, since very few data points are discarded during the warm-up phase [4]. Second, in our specific case of real-time audio transformation, this minimal warm-up requirement can **drastically speed up the initialization** of the system, making it a good candidate for testing real-time processing applications.

Note that although **NG-RC** is presented in the literature as mathematically equivalent to **ESN** [4], offering reduced computational complexity $\mathcal{O}(n)$, it is unlikely strong enough (in his architecture) to capture the rich dynamics we are looking for here. Indeed, it has not yet been compared to more capable models in our field. Currently, the only models known in the literature capable of handling such complex tasks rely on layered architectures, which provide greater richness in their memory and state dynamics for subsequent decoding [5, 16].

4.7.2 The Volterra Kernel

Finally in 2025 Grigorveya at al. [6]. Based on the theory of Reproducing Kernel Hilbert Spaces (RKHS), a complex mathematical vector family and the Volterra series representation, built a new framework within the reservoir computing family [6]. This model possesses the following properties:

- It can approximate any causal and time-invariant filter with the fading memory property [6].
- It is efficiently computable via explicit recursions [6].
- It is sequentially updatable, making it suitable for computing time series [6].
- It is a universal kernel, meaning it can approximate any nonlinear system with **ESP** [6].

This new paradigm is built upon the **Volterra reservoir kernel**. However, understanding the Volterra series and RKHS requires an advanced mathematical background and a dedicated state-of-the-art review. Furthermore, this approach is likely overkill for real-time audio distortion processing.

Finally, based on our understanding of its presentation [7], the computation of such a model requires a third-order tensor and a regression performed in an infinite-dimensional space. Therefore, we can reasonably assume that the computation of a single time step with a complexity of $\mathcal{O}(n^3)$. As previously noted in the literature regarding Volterra series [16], this increase in computational complexity severely limits their real-time applicability for audio processing at a 44.1 kHz sampling rate.

4.8 Bayesian Optimization of Hyper-parameters

Although it does not introduce a new Reservoir Computing model strictly speaking, the work of Yperman and Becker [25] is highly relevant to our search for the optimal architecture for real-time audio processing.

Their study describes an automated search method using a Gaussian process combined with Bayesian optimization to find the best hyperparameters of a reservoir [25]. This technique allows efficient and simultaneous optimization of multiple variables such as spectral radius, input scaling, number of neurons, and regularization parameter.

This approach has proven to be a powerful alternative to manual or random search strategies. By intelligently navigating the error surface to balance the exploitation of known good regions with the exploration of uncertain ones, Bayesian optimization significantly reduces the need for expert and converges toward an optimal set of hyperparameters [25].

The paper also mentions a Python library (Spearmint) specifically made for testing hyperparameters under the Bayesian optimization approach [25]. This can clearly be useful for later testing or implementation phases.

5 Conclusion

Now that we have covered the major evolutions in Reservoir Computing (RC) for real-time audio distortion tasks, we will examine the benchmarks obtained for the Mackey-Glass benchmark defined above.

Table 1: Reservoir Computing performance comparison on the Mackey-Glass benchmark

Model	Paper	Neuron number	Error (NRMSE)
Mackey-Glass $\tau = 17$ (Mildly chaotic)			
Band-pass ESN	[24]	3300	0.0065 [a]
Leaky-ESN	[13]	400	0.00012 [a]
Deep Time-Delay RC	[5]	1000	4.56×10^{-5} [b]
Mackey-Glass $\tau = 30$ (Wildly chaotic)			
Standard ESN	[11]	400	0.0718 [a]
Leaky-ESN	[13]	400	0.0320 [a]
Delay & Sum ESN	[11]	400	0.0200 [a]

[a] NRMSE for an 84-step ahead prediction (NRMSE84).
[b] NRMSE for a 34-step ahead prediction ($\Delta n = 34$).
[c] NRMSE for the reconstruction of the first 100 points.

The results demonstrate a clear progression in model performance for each successive generation.

This state-of-the-art review has traced the evolution of the **Reservoir Computing** paradigm, from its foundations to the most recent deep time-delay architectures. In response to our **first research question**, the literature demonstrates that the RC paradigm can successfully satisfy the requirements of nonlinear audio processing, if we move away from single-layer models [11, 14]. Even if **ESN** and **LSM** offer certain dynamical richness, their algorithmic complexity and memory limits their deployment on standard hardware [11, 16]. In this regard, the transition to **Delay-Line Reservoir Computing** (DLRC) represents a major breakthrough, reducing the computational complexity to $\mathcal{O}(N)$ and thereby enabling processing in real-time audio streams [2, 16].

Regarding our **second research question**, the comparative analysis designates layered architectures as the most performant solutions. The literature suggests that the **Cascaded DLRC** and **deepTRC** architectures are the best models for real-time audio distortion tasks [16, 5]. Specifically, the cascaded architecture proposed by Keuninckx et al. is to the best of our knowledge, currently the only model to have validated the emulation of a guitar amplifier at 44.1 kHz with an estimated latency of 0.11 ms (significantly below the 15 ms limit imposed by human auditory perception define above). The **deepTRC** model also emerges as a highly competitive candidate due to its ability to

structure a hierarchical memory across multiple timescales, while maintaining a strictly linear complexity [5].

Finally, although latest generation models such as the **NG-RC** promise a drastically reduced warm-up and training phase, their capacity to capture rich dynamics required by heavy audio distortion remains to be demonstrated. The models most suited for this task disposing both long and short timescales, offered by multiples layers. I personally question whether this approach truly permits the modeling of strong non-linear distortions, a point that requires future testing.

References

[1] Wikipedia, The Free Encyclopedia 2025. *Nonlinearity (disambiguation)*. Wikipedia, The Free Encyclopedia. Retrieved 2025-11-23 from https://en.wikipedia.org/wiki/Nonlinearity_(disambiguation)

[2] Lennert Appeltant, Maria C. Soriano, Guy Van der Sande, Jan Danckaert, Serge Massar, Joni Dambre, Benjamin Schrauwen, Claudio R. Mirasso, and Ingo Fischer. 2011. Information processing using a single dynamical node as complex system. *Nature Communications* 2, 1 (Sept. 2011), 468. doi:10.1038/ncomms1476

[3] Claudio Gallicchio, Alessio Micheli, and Luca Pedrelli. 2017. Deep reservoir computing: A critical experimental analysis. *Neurocomputing* 268 (2017), 87–99. doi:10.1016/j.neucom.2016.12.089

[4] Daniel J. Gauthier, Erik Bollt, Aaron Griffith, and Wendson A. S. Barbosa. 2021. Next-generation reservoir computing. *Nature Communications* 12, 1 (2021), 5564. doi:10.1038/s41467-021-25801-2

[5] M. Goldmann, F. Köster, K. Lüdge, and S. Yanchuk. 2020. Deep time-delay reservoir computing: Dynamics and memory capacity. *Chaos: An Interdisciplinary Journal of Nonlinear Science* 30, 9 (2020), 093124. doi:10.1063/5.0017974

[6] Lukas Gonon, Lyudmila Grigoryeva, and Juan-Pablo Ortega. 2025. Reservoir kernels and Volterra series. *IEEE Transactions on Neural Networks and Learning Systems* (September 2025). doi:10.1109/TNNLS.2024.3364344

[7] Lyudmila Grigoryeva. 2023. Lyudmila Grigoryeva: Reservoir kernels and Volterra series. https://www.youtube.com/watch?v=mta3ACJT9n4&t=4260s. Accessed: 1/04/2026.

[8] Marvyn Gulina. 2017. Utilisation d'un ordinateur réservoir pour décrypter un message transmis via la cryptographie par chaos. Sous la direction du Professeur Serge Massar.

[9] Marvyn Gulina. 2023. *On the approximations of the Koopman operator and applications to spectral identification of networks*. Ph. D. Dissertation. University of Namur, Faculty of Sciences, Department of Mathematics, Namur Institute for Complex Systems (naXys), Namur, Belgique. Advisor: Pr. Alexandre Mauroy. Co-Advisor: Pr. Timoteo Carletti. Jury: Pr. Joseph J. Winkin (President), Pr. Julien M. Hendrickx, Dr. rer. nat. Felix Dietrich..

[10] Georg Holzmann. 2009. Reservoir computing: a powerful black-box framework for nonlinear audio processing. In *Proceedings of the 12th International Conference on Digital Audio Effects (DAFx-09)*. 90–97.

[11] Georg Holzmann and Helmut Hauser. 2010. Echo state networks with filter neurons and a delay&sum readout. *Neural Networks* 23 (2010), 244–256. doi:10.1016/j.neunet.2009.07.004

[12] Herbert Jaeger. 2002. *A tutorial on training recurrent neural networks, covering BPPT, RTRL, EKF and the "echo state network" approach.* Technical Report GMD Report 159. German National Research Center for Information Technology.

[13] Herbert Jaeger. 2010. *The "echo state" approach to analysing and training recurrent neural networks – with an Erratum note.* Technical Report. Fraunhofer Institute for Autonomous Intelligent Systems. This is a corrected version of the original technical report: H. Jaeger (2001): The "echo state" approach to analysing and training recurrent neural networks, GMD Report 148, GMD. The corrections are given in a new Erratum section which is added as an Appendix..

[14] Herbert Jaeger and Harald Haas. 2004. Harnessing Nonlinearity: Predicting Chaotic Systems and Saving Energy in Wireless Communication. *Science* (2004). doi:10.1126/science.1091277

[15] Herbert Jaeger, Mantas Lukoševičius, Dan Popovici, and Udo Siewert. 2007. Optimization and applications of echo state networks with leaky-integrator neurons. *Neural Networks* 20 (2007). doi:10.1016/j.neunet.2007.04.016

[16] Lars Keuninckx, Jan Danckaert, and Guy Van der Sande. 2017. Real-time Audio Processing with a Cascade of Discrete-Time Delay Line-Based Reservoir Computers. *Cognitive Computation* 9, 3 (June 2017), 315–326. doi:10.1007/s12559-017-9457-5

[17] Wolfgang Maass, Thomas Natschläger, and Henry Markram. 2002. Real-time computing without stable states: A new framework for neural computation based on perturbations. *Neural Computation* 14, 11 (2002), 2531–2560. doi:10.1162/089976602760407955

[18] M. C. Mackey and L. Glass. 1977. Oscillation and chaos in physiological control systems. *Science* 197, 4300 (1977), 287–289.

[19] Marco Peixeiro. 2022. *Time Series Forecasting in Python.* Manning.

[20] Benjamin Schrauwen, Marion Wardermann, David Verstraeten, Jochen J. Steil, and Dirk Stroobandt. 2008. Improving reservoirs using intrinsic plasticity. *Neurocomputing* 71 (2008), 1159–1171. doi:10.1016/j.neucom.2007.12.020

[21] Jochen Triesch. 2005. A gradient rule for the plasticity of a neuron's intrinsic excitability. In *International Conference on Artificial Neural Networks.* Springer, 65–70.

[22] Nathan Trouvain, Luca Pedrelli, Thanh Trung Dinh, and Xavier Hinaut. 2020. ReservoirPy: an Efficient and User-Friendly Library to Design Echo State Networks. HAL Id: hal-02595026. Available at https://inria.hal.science/hal-02595026v2.

[23] Julien Vitay. n.d.. *Neurocomputing: Reservoir Computing*. Julien Vitay - University of Chemnitz. Retrieved 2026-02-12 from https://julien-vitay.net/lecturenotes-neurocomputing/4-neurocomputing/4-Reservoir.html

[24] Francis Wyffels, Benjamin Schrauwen, David Verstraeten, and Dirk Stroobandt. 2008. Band-pass reservoir computing. In *2008 International Joint Conference on Neural Networks (IJCNN 2008)*. IEEE, 3204–3209. doi:10.1109/IJCNN.2008.4634252

[25] Jan Yperman and Thijs Becker. 2016. Bayesian optimization of hyper-parameters in reservoir computing. *Neurocomputing* 213 (2016), 110–116. doi:10.1016/j.neucom.2016.05.048

[26] Shuai Zhang. 2024. *On the analyzing of bifurcation properties of the one-dimensional Mackey–Glass model by using a generalized approach*. doi:10.1002/mma.10381

[27] Udo Zölzer (Ed.). 2002. *DAFX: Digital Audio Effects*. John Wiley & Sons.